

Hyperparameter Optimization & Model Comparison

Objetivo (Markdown – curto e claro)

Goal: Optimize and compare two classification models — Random Forest and Logistic Regression — using systematic hyperparameter tuning.

The objective is to assess trade-offs between predictive performance and model interpretability.

```
In [1]: import pandas as pd
        from pathlib import Path
        import numpy as np

        from sklearn.model_selection import StratifiedKFold, GridSearchCV
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.linear_model import LogisticRegression

        from sklearn.metrics import roc_auc_score

In [2]: PROJECT_ROOT = Path.cwd()
        DATA_PATH = PROJECT_ROOT / "Data"

        df = pd.read_csv(DATA_PATH / "ObesityDataSet_raw_and_data_synthetic.csv")

In [3]: df["obesity_risk"] = df["NObeyesdad"].apply(
        lambda x: 0 if x in ["Insufficient_Weight", "Normal_Weight"] else 1
        )

        y = df["obesity_risk"]

In [4]: X = df.drop(
        columns=[
            "NObeyesdad",
            "obesity_risk",
            "Height",
            "Weight"
        ]
        )

In [5]: categorical_columns = X.select_dtypes(include="object").columns

        X_encoded = pd.get_dummies(
            X,
            columns=categorical_columns,
            drop_first=True
        )
```

```
X_encoded = X_encoded.reset_index(drop=True)
y = y.reset_index(drop=True)
```

Cross-Validation Strategy

```
In [6]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Random Forest — Search Space

```
In [7]: rf = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)

rf_param_grid = {
    "n_estimators": [100, 300],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}
```

GridSearchCV — Random Forest

```
In [8]: rf_search = GridSearchCV(
    estimator=rf,
    param_grid=rf_param_grid,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1
)

rf_search.fit(X_encoded, y)
```

```
Out[8]:
  ▸ GridSearchCV ⓘ ?
  ▸ best_estimator_: RandomForestClassifier
    ▸ RandomForestClassifier ?
```

Random Forest — Best Result

```
In [9]: print("=== Random Forest — Best Parameters ===")
print(rf_search.best_params_)

print("\nBest ROC-AUC (CV):")
print(rf_search.best_score_)
```

=== Random Forest – Best Parameters ===

```
{'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

Best ROC-AUC (CV):

0.9778789131493726

Logistic Regression — Search Space

```
In [10]: logreg = LogisticRegression(
            solver="liblinear",
            max_iter=1000,
            random_state=42
        )

logreg_param_grid = {
    "C": [0.01, 0.1, 1, 10],
    "penalty": ["l1", "l2"],
    "class_weight": [None, "balanced"]
}
```

GridSearchCV — Logistic Regression

```
In [11]: logreg_search = GridSearchCV(
            estimator=logreg,
            param_grid=logreg_param_grid,
            scoring="roc_auc",
            cv=cv,
            n_jobs=-1
        )

logreg_search.fit(X_encoded, y)
```

C:\Users\biauser\anaconda3\envs\edal\Lib\site-packages\sklearn\linear_model_logistic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and C=np.inf instead of penalty=None.

warnings.warn(

C:\Users\biauser\anaconda3\envs\edal\Lib\site-packages\sklearn\linear_model_logistic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio only.

warnings.warn(

```
Out[11]:
  ▸ GridSearchCV ⓘ ?
  ▸ best_estimator_: LogisticRegression
    ▸ LogisticRegression ?
```

Logistic Regression — Best Result

```
In [12]: print("=== Logistic Regression – Best Parameters ===")
print(logreg_search.best_params_)

print("\nBest ROC-AUC (CV):")
print(logreg_search.best_score_)
```

```
=== Logistic Regression – Best Parameters ===
{'C': 1, 'class_weight': 'balanced', 'penalty': 'l1'}
```

```
Best ROC-AUC (CV):
0.9153278077112482
```

Comparison Summary

Model Comparison Summary

AutoSklearn2 selected Random Forest as the best-performing model.

In the next step, I optimized both Random Forest and Logistic Regression.

While Random Forest achieved higher ROC-AUC, Logistic Regression offered greater interpretability, which is valuable in risk-screening contexts.

```
In [ ]:
```